Reproduce

Proving the toolchain works before trusting it with real material

This page is optional, and it is not the process

The process is: create the directories in `Directory_Creation.pdf`, install the toolchain in `Installation.pdf`, then open this folder in an agent platform and send it `Multi_Agent_Outline.md`. Nothing on this page replaces that. What it gives you is a way to check — cheaply, before committing a real lecture to a multi-hour agent run — that your machine and this kit actually agree with each other.

`Example_Deliverable_Directory/` is a completed run of this exact pipeline on a real lecture (`Example_Input_Directory/`'s source PDF), carried all the way through six rounds of review. It is frozen: do not write a new lecture into it, and do not edit its files to match a later tool change. It exists so you have a known-good reference to check your own environment against.

Checking the kit before you trust it

Command	What it establishes	Needs
`python3 tools/check_structure.py Example_Deliverable_Directory/*.pdf --alt Example_Deliverable_Directory/alt_text.json`	The example's tagged PDF still measures what it claims: reading inventory, MCID reachability, artifact audit, figure placement, table/list structure, MathML, alt text, metadata. This is the actual acceptance gate (Step 4d) — a conformance-checker pass alone is not.	python3, pymupdf
`verapdf -f ua1 --format text Example_Deliverable_Directory/*.pdf`	The example passes PDF/UA-1 conformance — the standard this pipeline currently targets (`Accessibility_Info.pdf` explains why UA-1 rather than UA-2).	veraPDF
`python3 tools/check_contrast.py --pdf Example_Deliverable_Directory/*.pdf`	Body text and figure ink clear WCAG's measured contrast thresholds, not just an eyeballed impression.	python3, pymupdf, pillow, numpy

If all three come back clean against the frozen example, your toolchain and this kit agree, and you're ready to point the agent at real material. If any of them disagree with what `Example_Deliverable_Directory/`'s own `accessibility/` folder already reports, something about your local toolchain differs from the one this kit was built and tested against — worth resolving before a real run, not after.

What a real run looks like

Once you send `Multi_Agent_Outline.md` to an agent with real material in `Input/`, expect this shape (the full detail is in the outline itself):

1. **Stage 0-1**: source pages are rendered and segmented into PNGs.
2. **Stage 2**: 3 independent reviewer sub-agents check the segments. REVISE loops back to Stage 1 until all 3 approve — this can take several rounds. That is expected, not a sign of trouble.
3. **Stage 3**: the approved segments assemble into one PDF per lecture.
4. **Stage 4**: the agent picks a reference textbook, transcribes to tagged LaTeX, and 5 physics-referee sub-agents plus an Accessibility Auditor review the compiled result. Again, expect REVISE rounds — the auditor is usually the last one still saying it, on purpose (see `Accessibility_Info.pdf` for why a physics reviewer cannot waive that verdict).
5. **Stage 5**: a self-contained hand-off package, built only after every gate above has closed.

A run that closes on the first round for every gate is unusual, not the target — the point of the review loops is that they catch things a single pass misses, and this project's own `PIPELINE_LOG.md` (if you have access to the source repository) documents specific defects that survived one clean approval and were only caught by a later, independent round.

If you want to rebuild the example yourself

`Example_Deliverable_Directory/` ships the already-transcribed `.tex` alongside the compiled PDF. Recompiling it with your own LuaLaTeX —

```
lualatex Example_Deliverable_Directory/*.tex
lualatex Example_Deliverable_Directory/*.tex
lualatex Example_Deliverable_Directory/*.tex
```

— is a toolchain check, not a pipeline run: it does not re-segment, re-review, or re-transcribe anything. Byte-identical output to the shipped PDF is not expected (fonts and metadata timestamps can differ across machines); **identical structure counts** when you run `check_structure.py` against your recompiled PDF is what actually confirms your toolchain reproduces the same tagging.